EX. NO: 4

IMPLEMENTATION OF DIGITAL SIGNATURE STANDARD (DSA)

AIM:

To write a Java program to implement the Digital Signature Standard (DSA) algorithm.

DESCRIPTION:

The Digital Signature Algorithm (DSA) lets a signer prove authorship of a message without revealing their private key, and lets anyone verify that proof using only the signer's public key. DSA relies on a set of shared public parameters (a prime p, a prime divisor q of p-1, and a generator g of the order-q subgroup), a private key x (random, less than q) and the corresponding public key y = g^x mod p. To sign a message hash h, the signer picks a random per-message secret k, computes r = (g^k mod p) mod q and s = k^-1(h + x*r) mod q; the signature is the pair (r, s). A verifier recomputes v from the public key, g, r, s and h, and accepts the signature only if v equals r.

ALGORITHM:

STEP-1: Generate the global public parameters: a prime p, a prime divisor q of (p-1), and a generator g of the order-q subgroup mod p.

STEP-2: The signer picks a private key x (random, 0 < x < q) and computes the public key y = g^x mod p.

STEP-3: For a given message hash h, pick a random per-message secret k and compute r = (g^k mod p) mod q.

STEP-4: Compute the signature component s = k^-1 * (h + x*r) mod q. The signature is the pair (r, s).

STEP-5: To verify: compute w = s^-1 mod q, u1 = h*w mod q, u2 = r*w mod q, and v = ((g^u1 * y^u2) mod p) mod q.

STEP-6: If v equals r, the signature is valid; otherwise it is a forgery.

PROGRAM: (Java)

```
import java.util.*;
import java.math.BigInteger;

class DSAAlg {
    final static BigInteger one = new BigInteger("1");
    final static BigInteger zero = new BigInteger("0");

    public static BigInteger getNextPrime(String ans) {
        BigInteger test = new BigInteger(ans);
        while (!test.isProbablePrime(99))
        {
            test = test.add(one);
        }
        return test;
    }

    public static BigInteger findQ(BigInteger n)
    {
        BigInteger start = new BigInteger("2");
        while (!n.isProbablePrime(99))
        {
            while (!((n.mod(start)).equals(zero)))
            {
                start = start.add(one);
            }
            n = n.divide(start);
        }
        return n;
    }

    public static BigInteger getGen(BigInteger p, BigInteger q, Random r)
    {
        BigInteger h = new BigInteger(p.bitLength(), r);
```

```
        h = h.mod(p);
        return h.modPow((p.subtract(one)).divide(q), p);
    }

    public static void main (String[] args) throws Exception
    {
        Random randObj = new Random();
        BigInteger p = getNextPrime("10600");
        BigInteger q = findQ(p.subtract(one));
        BigInteger g = getGen(p,q,randObj);

        System.out.println("\nSimulation of Digital Signature Algorithm\n");
        System.out.println("Global public key components are:\n");
        System.out.println("p is: " + p);
        System.out.println("q is: " + q);
        System.out.println("g is: " + g);

        BigInteger x = new BigInteger(q.bitLength(), randObj);
        x = x.mod(q);
        BigInteger y = g.modPow(x,p);

        BigInteger k = new BigInteger(q.bitLength(), randObj);
        k = k.mod(q);
        BigInteger r = (g.modPow(k,p)).mod(q);

        BigInteger hashVal = new BigInteger(p.bitLength(), randObj);
        BigInteger kInv = k.modInverse(q);
        BigInteger s = kInv.multiply(hashVal.add(x.multiply(r)));
        s = s.mod(q);

        System.out.println("\nSecret information:\n");
        System.out.println("x (private) is: " + x);
        System.out.println("k (secret) is: " + k);
        System.out.println("y (public) is: " + y);
        System.out.println("h (message hash) is: " + hashVal);

        System.out.println("\nGenerating digital signature:\n");
        System.out.println("r is: " + r);
        System.out.println("s is: " + s);

        BigInteger w = s.modInverse(q);
        BigInteger u1 = (hashVal.multiply(w)).mod(q);
        BigInteger u2 = (r.multiply(w)).mod(q);
        BigInteger v = (g.modPow(u1,p)).multiply(y.modPow(u2,p));
        v = (v.mod(p)).mod(q);

        System.out.println("\nVerifying digital signature (checkpoints):\n");
        System.out.println("w is: " + w);
        System.out.println("u1 is: " + u1);
        System.out.println("u2 is: " + u2);
        System.out.println("v is: " + v);

        if (v.equals(r))
            System.out.println("\nSuccess: digital signature is verified! r = " + r);
        else
            System.out.println("\nError: incorrect digital signature");
    }
}
```

SAMPLE OUTPUT:

```
Simulation of Digital Signature Algorithm

Global public key components are:

p is: 10601
q is: 53
g is: 566

Secret information:

x (private) is: 49
k (secret) is: 9
y (public) is: 3848
h (message hash) is: 10644

Generating digital signature:

r is: 36
s is: 36

Verifying digital signature (checkpoints):

w is: 28
u1 is: 13
u2 is: 1
v is: 36

Success: digital signature is verified! r = 36

=== Code Execution Successful ===
```

RESULT:

Thus the Digital Signature Standard (DSA) algorithm was implemented successfully using Java and the generated signature was verified correctly.